Assignment 2: Data

Ethan Chung
University of Hawaii
ECE 405: Intro to Large-Scale AI Systems
[redacted]@hawaii.edu

April 3, 2026

Contents

Filtering Common Crawl **2**
Problem (look_at_cc): 4 points . . . 2
Problem (encode_text): 3 points . . . 3
Problem (language_identification): 6 points . . . 3
Problem (mask_pii): 3 points . . . 4
Problem (harmful_content): 6 points . . . 5
Problem (gopher_quality_filters): 3 points . . . 6
Problem (quality_classifier): 15 points . . . 6

Deduplication **7**
Problem (exact_deduplication): 3 points . . . 7
Problem (minhash_deduplication): 8 points . . . 7

Appendix **7**
Code Availability . . . 7

Filtering Common Crawl

Problem (look_at_cc): 4 points

(a) Look at the very first web page of the WARC file. What is its URL? Is it still accessible? Can you tell what the page seems to be about by looking at the raw HTML?

The URL is `http://0371rykj.com/ipfhsb/34.html` and the page is no longer accessible (stuck on "Connecting to..."). It is not immediately clear what this page is about as most of the content is in Chinese, however, the raw HTML mentions "product" a few times, so it could be a storefront. After translating some text into English, this does indeed seem to be the case: "Constant Temperature and Humidity Test Chamber [...] Product Application [...] essential testing equipment."

(b) Let's now look at the corresponding WET file. Notice that much of the extracted text is reminiscent of the HTML structure, and not actually the page's main content. Are there parts of the text you see that you think should have been filtered out by the extractor? Think about the quality of this text as training data: what might go wrong in training a model on text that looks like this? Conversely, what useful information can a model potentially extract from this page?

Some content that should have been filtered out includes navigation keywords, breadcrumbs, and boilerplate repetitive elements that are only useful for navigating the website but don't contain any important semantic meaning. If this data is given to a model, it could result in poor generation quality, as a lot of the text is fragmented from navigation or buttons. It also contains pinyin for specific characters in the sentences, which may not be desirable and could lead the model to generate text with these artifacts. There is still a lot of useful information, however, like product descriptions, contact info, and technical specifications.

(c) What makes a good training example is highly contextual. Describe an application domain for which this example might be useful to have in the training data, and one where it might not be.

The data contains great technical information, hardware models, and manufacturing brands, which could help with getting insight on specialized testing equipment, technical specifications, and industrial hardware. However, if the goal is training a creative writing model, these details are not useful, and the fragmented text from the boilerplate may reduce the model's ability to generate grammatically correct creative text.

(d) Look through 25 more WET records. For each record, very briefly comment on the document's language, the domain name, what type of page it is, etc. How many examples does it take until you see what you'd deem a "high-quality" webpage?

1. `10www.chinatikfans.com`: Chinese - Forum/Bulletin Board profile or blog page.
2. `13.usnccm.org`: English - Conference homepage (US National Congress on Computational Mechanics).
3. `176.utchat888.com`: Chinese (Traditional) - Adult/dating video chat room site.
4. `176766.cn`: Chinese - Adult content / spam page.
5. `183.136.223.153:8060`: Chinese - Likely a spam/bot generator site.
6. `189.166.42.203:8016`: Chinese - Login portal for some internal system/database.
7. `1m31m3.com`: Chinese - Gaming portal / detail page for a game.
8. `1stsourceonline.com`: English - E-commerce category page (office supplies/pens).
9. `2012.hackitoergosum.org`: English - Blog author page for a hacker/security conference.

10. `2014.12306-6.com`: Chinese - Tag index page, looks like a blog or article aggregator.
11. `2016.subudworldcongress.org`: English - Blog/news archive for a world congress site.
12. `202.109.130.63:8090`: Chinese - Login portal or Single Sign-On page.
13. `210.12.219.78`: Chinese - Bulletin Board System (BBS) forum post.
14. `25b5a.com`: Chinese - Image hosting or content sharing page.
15. `3-4.ru:80`: Russian - Directory or listing page for music and theater.
16. `315.hainan.gov.cn`: Chinese - Government administrative guide or news article.
17. `3721yy.com`: Chinese - Tag index page for a video or entertainment site.
18. `38j6d.com`: Chinese - Question/Answer or article page.
19. `3dprinters-usa.com`: English - E-commerce product page for 3D printers.
20. `3t.am`: English/Armenian - E-commerce product page missing full description.
21. `43.qiqizhuzhu.com`: Chinese - Unknown index page or empty template.
22. `45.115.144.137`: Chinese - Forum threads.
23. `5.newtianya.cn`: Chinese - Article or forum post page.
24. `50th.cahb.org.cn`: Chinese - Contact us page for an organization's 50th anniversary site.
25. `51.la`: Chinese - Statistical analytics or reporting dashboard.

It took about 2 examples (the USNCCM conference site) to see what could be considered a "high-quality" webpage with clean informational text, but a majority of the others were e-commerce sites, login portals, or low-quality forum thread/bulletin pages.

Problem (encode_text): 3 points

(a) Write a function that extracts text from a byte string containing raw HTML.

Implemented in cs336_data/extract_text.py

(b) Run your text extraction function on a single WARC file. Compare its output to the extracted text in the corresponding WET file. What differences and/or similarities do you notice? Which extraction seems better?

The text extracted by our function using Resiliparse preserves structural elements from the HTML, such as list bullets, spacing, and indentations, whereas the extracted text in the WET files contains flat text stripped entirely of structural whitespace and list markers. Our function also leaves behind unparsed HTML tags when the markup is malformed occasionally. The WET file extraction seems better for tasks that require clean text since it is more uniform and has less formatting artifacts.

Problem (language_identification): 6 points

(a) Write a function that will take a Unicode string and identify the main language that is present in this string.

Implemented in cs336_data/language_identification.py

(b) The behavior of language models at inference time largely depends on the data they were trained on. As a result, issues in the data filtering pipeline can result in problems downstream. What issues do you think could arise from problems in the language identification procedure? In a higher-stakes scenario (such as when deploying a user-facing product), how would you go about mitigating these issues?

If the language identification procedure incorrectly filters out data, the trained model might generate responses in non-english languages during inference, or will end up with mixed-language gibberish. In a high-stakes product, I would mitigate these issues by utilizing an ensemble of language identification models to improve robustness. I would also employ human-in-the-loop review to review the model's identifications, however likely randomly-sampled as it would be unfeasible to review absolutely everything.

(c) Run your language identification system on text extracted from the WARC files (via your previously-implemented text extraction function). Manually identify the language in 20 random examples and compare your labels with the classifier predictions. Report any classifier errors. What fraction of documents are English? Based on your observations, what would be a suitable classifier confidence threshold to use in filtering?

The classifier performed accurately across 20 random examples, with the only obvious error occurring because the model notably struggles to classify extremely short texts (e.g., example 15). Overall, it identified 10% (2/20) of the sample as primarily English. Prediction scores for English text typically hovered around 0.80, while Chinese texts frequently reached 0.95. Based on these observations, a confidence threshold between 0.70 and 0.80 is suitable for safely filtering documents.

— Example 3 — Language: en (Probability: 0.8092) Text Sample: Skip to main content Home USNCCM 13 13th US National Congress on Computational Mechanics San Diego, CA July 26-30, 2015 Main menu • Home Congress Information • Important Dates • Organization...

— Example 4 — Language: zh (Probability: 0.9952) Text Sample: 打影音聊天室依照站分定未十八者勿24小客服: 02-27654066 真: 02-27659081 我| 加入| 首”o 道排行榜恭喜三月份消排行前十名得免 No.1 LV75699** -10,000 No.2 LV28160** -9,000 No.3 LV72247** -8,0...

— Example 15 — Language: da (Probability: 0.2814) Text Sample: 404 Not Found nginx...

See evaluations in scripts/out_language_identification.txt

Problem (mask_pii): 3 points

(a) Write a function to mask out emails. Your function will take a string as input, and replace all instances of email addresses with the string "|||EMAIL_ADDRESS|||".

Implemented in cs336_data/mask_pii.py

(b) Write a function to mask out phone numbers. Your function will take a string as input, and replace all instances of phone numbers with the string "|||PHONE_NUMBER|||". Doing this reliably can be extremely challenging, as phone numbers might be written in an extremely diverse set of 5formats, but you should try to capture at least the most common phone number formats used in the United States, and be robust to minor syntactic deviations.

Implemented in cs336_data/mask_pii.py

(c) Write a function to mask out IP addresses. For this problem, it is enough to focus on IPv4 addresses (4 numbers up to 255 separated by points). Your function will take a string as input, and replace all instances of IP addresses with the string "|||IP_ADDRESS|||".

Implemented in cs336_data/mask_pii.py

(d) What problems do you think might arise downstream in a language model when these filters are naïvely applied on the training set? How might you mitigate these issues?

One such problem could be the model losing understanding of phone numbers / email addresses and could generate responses to such queries with the placeholder instead. Our models might lose the ability to comprehend calendar dates (e.g., `2022-9 1234 5678`... masked as a phone number) or quantities/numeric IDs that coincidentally match some phone formats. To mitigate these issues, basic regex patterns should be combined with contextual checks (e.g., requiring preceding keywords like "Tel:") or replaced with Named Entity Recognition (NER) models to verify PII before redaction.

(e) Run your PII masking functions on text extracted from the WARC files (via your previously-implemented text extraction function). Look through 20 random examples where a replacement was made; give some examples of false positives and false negatives.

After running the PII masking functions on text extracted from the WARC records, reviewing 20 random examples showed generally good performance on standard contact blocks, successfully masking emails like `bambus@i.ua` and structured numbers like `+1 (218) 654-2022`. However, false positives were common with the phone regex matching only partial parts of a number (e.g., `+86-13311891208` to `+86-|||PHONE_NUMBER|||`), or matches random strings of numbers such as pricing like in this example, which seems to be showing the stock price:

```
Original:  ...跟金
新市ETF
最大成交
ＳＰＤＲ金
2,379.000
02840
+12.000 (0.507%)
Masked:  ...跟金
新市ETF
最大成交
ＳＰＤＲ金
2,|||PHONE_NUMBER|||0
+12.000 (0.507%)
```

See evaluations in scripts/out_mask_pii.txt

Problem (harmful_content): 6 points

(a) Write a function to detect NSFW content.

Implemented in cs336_data/language_identification.py

(b) Write a function to detect toxic speech.

Implemented in cs336_data/language_identification.py

(c) What problems do you think might arise downstream in a language model when these filters are applied to create the training set? How might you mitigate these issues?

When these filters (trained mainly on English datasets like Jigsaw) are applied, they may disproportionately falsely classify text from other languages or out-of-domain contexts, while completely failing to catch harmful content in languages they haven't seen. To mitigate these issues, we should use multilingual training data for the classifiers, employ context-aware transformer models (perhaps some variant of BERT), and again incorporate human-in-the-loop auditing of randomly-sampled text to check the accuracy.

(d) Run your harmful content filters on text extracted from the WARC files (via your previously-implemented text extraction function). Look through 20 random examples and compare the classifier predictions to your own judgments. Report any classifier errors. What fraction of documents are harmful? Based on your observations, what would be suitable classifier confidence threshold(s) to use in filtering?

Evaluating the 20 random examples revealed a critical vulnerability in the classifier: a complete failure to detect non-English harmful content. Based on manual review, 35% (7/20) of the documents contained explicit Chinese NSFW material, such as pornography and adult webcam sites. However, the classifier generated false negatives for every single one, confidently labeling them as "non-nsfw" with scores ranging from 0.94 to 1.00. As a result, there is no suitable classifier confidence threshold for this filtering, so either the entire model will need to be switched with one that had multilingual capabilities, or the language model should reject all non-English languages.

> NSFW: non-nsfw (0.9863)
>
> Toxic: non-toxic (0.9967)
>
> Original: 打影音聊天室依照站分定未十八者勿24小客服: 02-27654066 真: 02-27659081 我 | 加入 | 首 盟道排行榜恭喜三月份消 排行前十名 得免 No.1 LV75699** -10,000 No.2 LV28160** -9,000
>
> Translated (Google): Masturbation Zone Video & Audio Chat Rooms In accordance with website content rating regulations, persons under the age of 18 are prohibited from browsing this site. 24-Hour Customer Service Hotline: 02-27654066 | Fax: 02-27659081 Contact Us | Sign Up | Home Money-Making Affiliate Program Channel Rankings Congratulations to the Top 10 Spenders of March! You have won free bonus points No. 1: LV75699** — 10,000 Bonus Points | No. 2: LV28160** — 9,000 Bonus Points

See evaluations in scripts/out_harmful_content.txt

Problem (gopher_quality_filters): 3 points

(a) Implement (at least) the subset of the Gopher quality filters as described above.

Implemented in cs336_data/language_identification.py

(b) Run your rule-based quality filter on text extracted from the WARC files (via your previously-implemented text extraction function). Look through 20 random examples and compare the filter predictions to your own judgment. Comment on any cases where the quality filters differ from your judgments.

A vast majority (16/20) fail the heuristic rules. Most fail because they are not English content, meaning standard space-based tokenization doesn't work. For instance, high-quality Chinese text (like Example 12) fails because the lack of spaces causes entire sentences or paragraphs to be treated as single, massive words, heavily violating the expected word-length heuristics (e.g., mean word length between 3 and 10 characters). Interestingly, non-Latin scripts that do use spaces (like the Greek text in Examples 10 and 11) successfully passed the filters. This demonstrates that these structural heuristics are highly brittle for non-space-delimited languages, suggesting they should only be applied downstream of a language identification filter to prevent inadvertently discarding high-quality multilingual data.

See evaluations in scripts/out_gopher_quality_filters.txt

Problem (quality_classifier): 15 points

(a) Train a quality classifier that, given text, returns a numeric quality score.

From an initial sample of 10,000 high-quality Wiki texts, we retained 2,494 viable examples after accounting for failed page scrapes and Gopher filtering. To maintain a balanced training distribution, we paired these with an equal number (2,494) of low-quality examples sampled from the Common Crawl dataset (with no Gopher filtering applied). The model was trained for 100 epochs at a learning rate of 0.1, achieving a final training loss of 0.078928.

See evaluations in scripts/out_quality_classifier.txt

Implemented in scripts/train_quality_classifier.py

(b) Write a function that labels a page as high or low-quality, and provides a confidence score in the label.

To evaluate the model's performance, we sampled 20 random texts: 10 from Common Crawl and 10 from Wikipedia (specifically, texts filtered out by Gopher to prevent train-test leakage). The classification accuracy between Common Crawl and Wikipedia was 90%, demonstrating the model's strong ability to distinguish between low- and high-quality data. Furthermore, the two Wikipedia samples misclassified as Common Crawl (Examples 5 and 20) were reasonable errors. Both texts consisted primarily of website navigation boilerplate and CAPTCHA verification messages, making their "low-quality" predictions logically sound and arguably beneficial for dataset filtering.

Implemented in cs336_data/quality_classifier.py

Deduplication

Problem (exact_deduplication): 3 points

(a) Write a function that takes a list of paths to input files and performs exact line deduplication on them. It should first count the frequency of each line in the corpus, using a hash to reduce memory, and then rewrite each file by only keeping its unique lines.

Implemented in cs336_data/exact_duplication.py

Problem (minhash_deduplication): 8 points

(a) Write a function that takes a list of paths to input files and performs fuzzy document deduplication with minhash and LSH. In particular, your function should compute minhash signatures for each document in the provided list of paths, use LSH with the provided number of bands to identify candidate duplicates, and then compute the true ngram Jaccard similarity between candidate duplicates and remove those that exceed a given threshold. To improve recall (following Penedo et al., 2023), normalize the text before computing minhash signatures and/or comparing Jaccard similarity by lowercasing, removing punctuation, normalizing whitespaces, and removing accents, and applying NFD unicode normalization.

Implemented in cs336_data/minhash_deduplication.py

Appendix

Code Availability

All code mentioned in this report is available at github.com/echung32/ece405-assignment2-data